

HW 0

Sam Ly

January 29, 2026

Chapter 1

1. Markdown, Jinja2, Makefile, SASS, CSS, HTML.
2.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello ,␣world!");  
    }  
}
```
3.

```
#include <stdio.h>  
  
int main() {  
    printf("Hello ,␣world!");  
}
```

Chapter 2

1. I picked Haskell. For the compiler, they actually use Haskell! The language is implemented in itself. This is to be expected since Haskell is pretty popular for compilers.
2. Just-in-time compilation is extremely difficult to do well. This is because JIT requires extremely deep knowledge of the language and implementation. Improperly implement JIT, even if correct, can actually end up costing performance rather than gaining.
3. Interpreted languages can be easier to develop with since you can type straight into the REPL and see the effects. However for performance, compiled language has them beat. By allowing both an interpreted and compiled runtime option, Lisp can get the best of both worlds.

Chapter 3

1. One potential edge case that wasn't defined in the book is nested classes. For example, if you were to make a Linked List class, the Node class may be an inner class that doesn't need to exist outside of the Linked List class scope.

Attempting to use inner classes actually leads to a compile error:

```
class LinkedListStack {  
    class Node {  
        init(data, next) {  
            self.data = data;  
            self.next = next;  
        }  
    }  
}
```

```

    }

    init() {
        self.head = nil;
    }
    // ...
}

```

[line 2] Error at 'class ': Expect method name.

The correct way to implement this is to write:

```

class Node {
    init(data, next) {
        this.data = data;
        this.next = next;
    }
}

class LinkedListStack {

    init() {
        this.head = nil;
    }

    push(data) {
        if (this.head == nil) {
            this.head = Node(data, nil);
        }
        else {
            var next = Node(data, this.head);
            this.head = next;
        }
    }

    pop() {
        if (this.head == nil) {
            return nil;
        }
        else {
            var out = this.head;
            this.head = this.head.next;
            return out;
        }
    }
}

```

This behaves as I expect since Lox is meant to be a simple language. Nested classes introduce a lot of complexity into the implementation.

2. One unanswered question I have relates to the idea of “truthy-ness”. In languages like Python and JavaScript, certain can evaluate to values when used as the predicate for an “if” statement. Also, these two languages even have different implementations of truthy-ness!

Example in JS:

```

const truthy = []
const falsy = null;

if (truthy) console.log("[] is true!");
if (!falsy) console.log("null is false!");

// [] is true!
// null is false!

```

Example in Python:

```

empty_arr = []
arr = [1,2,3]

if not empty_arr:
    print("empty_arr is false!")

if arr:
    print("arr is true!")

# empty_arr is true!
# arr is true!

```

3. The most important feature that is missing in my opinion is the ability to define return types for functions. I am a big fan of type safety, but I am aware that type safety is quite difficult to implement